



QATIP Intermediate Azure Lab04

Terraform Modules

Contents

Lab Objectives.....	1
Before you begin.....	1
Instructions	2
Review	4
Demonstration Deployment.....	5

Lab Objectives

- Develop reusable Terraform modules for Azure Virtual Networks.
- Parameterize the module using variables.
- Deploy multiple virtual networks by reusing the module with different parameters.
- Utilize module outputs in the root module to create dependent resources.

Before you begin

- In the IDE terminal pane, enter the following command...
cd c:\azure-tf-int\labs\04
- This shifts your current working directory to **labs\04**. Ensure all commands are executed in this directory
- Close any open files and use the Explorer pane to navigate to and open the **labs\04** folder.

The Challenge

Using **c:\azure-tf-int\labs\04** as your root directory, you are to create a reusable module structure to deploy 2 Azure virtual networks into resource group RG1, located in East US. As you deploy the module, expose outputs from it and use these, from the root module, to create a subnet on each network.

Key Requirements

- Create **main.tf**, **variables.tf**, and **outputs.tf** files as part of the module
- Deploy at least two virtual networks using a module structure
- Pass output values from the module back to the root configuration
- Use the outputs from the module to create a subnet in each network

Instructions

Set Up the Project Directory

1. Ensure you have moved to the lab directory..

```
cd c:\azure-tf-int\labs\04
```

2. Create a subdirectory for the module

```
mkdir modules\vnet
```

Create the VNet Module

3. Navigate to the new vnet directory and create the following files:

main.tf:

```
resource "azurerm_virtual_network" "vnet" {  
  name = var.vnet_name  
  address_space = var.address_space  
  location = var.location  
  resource_group_name = var.resource_group_name  
}
```

variables.tf:

```
variable "vnet_name" {}
```

```
variable "address_space" { type = list(string)}  
variable "location" {}  
variable "resource_group_name" {}
```

outputs.tf:

```
output "vnet_id" {  
  value = azurerm_virtual_network.vnet.id  
}
```

```
output "vnet_name" {  
  value = azurerm_virtual_network.vnet.name  
}
```

```
output "vnet_address_space" {  
  value = azurerm_virtual_network.vnet.address_space  
}
```

Review and update the Root module Configuration

4. In **c:\azure-tf-int\labs\04**, update line 2 of **main.tf** with your subscription id

Initialize, Review, Apply and Verify

5. Run ..

```
terraform init  
terraform plan  
terraform apply --auto-approve
```

Terraform state list

Verify in the Azure Portal.

Step 6: Clean Up Resources

```
terraform destroy --auto-approve
```

Review

This configuration, though simplistic, is a well-structured solution to the given challenge. It adheres to best practices by utilizing **reusable modules** to deploy two Azure Virtual Networks (vnet1 and vnet2) within a single resource group (RG1), located in **East US**. The root module orchestrates the deployment by calling the virtual network module twice—once for each VNet—while ensuring outputs from the module are exposed and consumed to create subnets within each network.

Module-Based Approach

A key requirement is to create a **reusable module structure**. This is accomplished by defining a vnet module stored under `c:\azure-tf-int\labs\04\modules\vnet`. Instead of defining virtual networks directly in the root module, this module encapsulates the logic for provisioning a VNet with configurable parameters, such as:

- **Name (vnet_name)** – Allows different VNets to be created using the same module.
- **Address Space (address_space)** – Ensures each network has a unique IP range.
- **Resource Group (resource_group_name)** – Associates the VNets with the specified resource group.
- **Location (location)** – Ensures all resources are deployed in East US.

By using **module instantiation**, the root module deploys two virtual networks (vnet1 and vnet2) by calling the vnet module twice, passing different parameters for each instance.

Exposing and Utilizing Module Outputs

A critical part of the challenge was to expose outputs from the module and utilize them in the root module to create subnets. This is achieved through the output block within the vnet module, which ensures that the virtual network's name and ID are accessible after deployment. The root module then references these outputs to create subnets within each network.

Scalability and Reusability

By structuring the solution with modules, this approach is highly scalable. If additional virtual networks are required, they can be easily instantiated without modifying the module itself. Additionally, the structure promotes code reuse, making it applicable for future scenarios where multiple VNets need to be deployed across different resource groups or locations.

Demonstration Deployment

Overview

1. This demonstration walks you through a more complex use of modules to deploy Virtual Networks (VNets), Network Security Groups (NSGs), and VNet Peering. The goal is to further highlight the benefits of dynamic and reusable infrastructure by leveraging Terraform modules and outputs.
2. In your IDE, navigate to **c:\azure-tf-int\labs\04\demo**
3. Review the provisioned files
4. Update line **2** of **demo\main.tf** with your **subscription id**

Understanding the flow logic

To efficiently provision multiple VNets with multiple subnets, a single module block (“vnets”) is used. Values to be passed down to the module are first read from variables, defined in **variables.tf** and populated in **terraform.tfvars**

```
module "vnets" {  
  for_each      = var.vnets  
  source        = "../modules/vnet"  
  vnet_name     = each.value.name  
  address_space = each.value.address_space  
  location      = each.value.location  
  resource_group_name = azurerm_resource_group.example.name  
  subnets      = [for k, v in var.subnets[each.key] : {  
    name = k, address_prefix = v }]  
}
```

The “**for_each = var.vnets**” loop will invoke the creation of as many VNets as there are objects in **var.vnets**, in this case 2’ For each iteration, values are passed into the module, drawn from **var.vnets**, relative to the current iteration..

```
vnets = {
  vnet1 = {
    name          = "vnet-01"
    address_space = ["10.0.0.0/16"]
    location       = "East US"
  }

  vnet2 = {
    name          = "vnet-02"
    address_space = ["10.1.0.0/16"]
    location       = "West Europe"
  }
}
```

So, the first iteration will create a Vnet named “vnet-01” with an address range of 10.0.0.0/16 in East US.

var.subnets is a nested map. The outer map uses VNet names as keys and the inner map contains subnets.

```
subnets = {
  vnet1 = {
    apps = "10.0.1.0/24"
    data = "10.0.2.0/24"
  }

  vnet2 = {
    apps = "10.1.1.0/24"
    data = "10.1.2.0/24"
  }
}
```

When calling the module, we only pass the subnets that belong to that specific VNet. This is achieved through **list comprehension**, used here in the format:

```
subnets =
  [for k,v in var.subnets[each.key]:{name=k,address_prefix=v}]
```

each.key gets the current VNet’s key (vnet1 or vnet2). The key value used each time is derived from the main **for_each** loop

var.subnets[each.key] Extracts only the subnets for that Vnet. List conversion converts the inner map into a list of objects ({ name, address_prefix })

So, the first iteration will create a Vnet named “vnet-01” with an address range of 10.0.0.0/16 in East US and subnets apps (10.0.1.0/24) and data (10.0.2.0/24)

In the VNnets module, the variables, defined in variables.tf are populated with the values passed down from the root module. If no value is passed then a default value is used or a prompt is issued. These values are then used by the modules’ **main.tf** to deploy the VNet and subnets.

```
variable "resource_group_name" {}
variable "vnet_name" {}
variable "address_space" {
  type = list(string)
}
variable "location" {}

variable "subnets" {
  type = list(object({
    name          = string
    address_prefix = string
  }))
  default = []
}

resource "azurerm_virtual_network" "vnet" {
  name                = var.vnet_name
  address_space       = var.address_space
  location            = var.location
  resource_group_name = var.resource_group_name
}

resource "azurerm_subnet" "subnet" {
  for_each = { for s in var.subnets : s.name => s }

  name                = each.value.name
  resource_group_name = var.resource_group_name
  virtual_network_name = azurerm_virtual_network.vnet.name
  address_prefixes     = [each.value.address_prefix]
}
```

Finally, selective Vnet and subnet parameters are passed from the module to the root module as needed through outputs..

```
output "vnet_details" {
  value = {
    id = azurerm_virtual_network.vnet.id
  }
}

output "subnets" {
  value = { for s in azurerm_subnet.subnet : s.name => s.id }
}
```

So, in summary..

- A single module block provisions all VNets
- Looping through the **var.vnets** variable using **for_each** creates VNets dynamically.
- Subnets are stored as a map of maps variable, grouped under their respective VNets.
- List comprehension ([for k, v in var.subnets[each.key] ...]) ensures only the correct subnets are assigned to each VNet.
- The VNet module receives these values, creating subnets dynamically using **for_each**.

5. Run...

```
cd c:\azure-tf-int\labs\04\demo
```

```
terraform init
```

```
Initializing the backend...
Initializing modules...
- vnets in modules\vnet
Initializing provider plugins...
- Reusing previous version of hashicorp/azurerm from the dependency lock file
- Installing hashicorp/azurerm v4.18.0...
- Installed hashicorp/azurerm v4.18.0 (signed by HashiCorp)
```

Note how the vnet module is initialized. Also note that a modules folder is created under .terraform...

```
└─ .terraform
  └─ modules
      └─ modules.json
```

The file modules.json is created and identifies the module, its source and source directory. If the module was remote then a copy of the module files would also be pulled and saved locally...

```
labs > 04 > demo > .terraform > modules > {} modules.json > ...
1 [{"Modules":[{"Key":"","Source":"","Dir":"."}, {"Key":"vnets","Source":"./modules/vnet","Dir":"modules/vnet"}]}
```

6. Run...

terraform apply --auto-approve

terraform state list

8. Run terraform console

```
PS C:\azure-tf-int\labs\04\demo> terraform console
> module.vnets
{
  "vnet1" = {
    "subnets" = {
      "apps" = "/subscriptions/28675585-9c0c-4857-9c0c-4857-9c0c-4857-9c0c-4857-9c0c-4857-9c0c"
      "data" = "/subscriptions/28675585-9c0c-4857-9c0c-4857-9c0c-4857-9c0c-4857-9c0c-4857-9c0c"
    }
    "vnet_details" = {
      "id" = "/subscriptions/28675585-9c0c-4857-9c0c-4857-9c0c-4857-9c0c-4857-9c0c-4857-9c0c"
      "location" = "eastus"
      "name" = "vnet-01"
      "resource_group" = "RG1"
    }
  }
}
```

11. The Vnets created and their associated subnets are now driven by input data rather than explicit, per-resource coding. Test this by reviewing and editing **terraform.tfvars**, adding a third subnet to vnet2...

12. Run...

terraform plan

Version 1.1.0

09/06/26

mcgreen@qa.com

terraform apply --auto-approve

```
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.  
PS C:\azure_tf\int\labs\04\demo> terraform state list
```

terraform state list

```
azurerm_resource_group.example  
module.vnets["vnet1"].azurerm_subnet.subnet["apps"]  
module.vnets["vnet1"].azurerm_subnet.subnet["data"]  
module.vnets["vnet1"].azurerm_virtual_network.vnet  
module.vnets["vnet2"].azurerm_subnet.subnet["apps"]  
module.vnets["vnet2"].azurerm_subnet.subnet["data"]  
module.vnets["vnet2"].azurerm_subnet.subnet["mgmt"]  
module.vnets["vnet2"].azurerm_virtual_network.vnet
```

13. Verify the new subnet exists in the Portal..

☐	Name ↑	IPv4
☐	apps	10.1.1.0/24
☐	data	10.1.2.0/24
☐	mgmt	10.1.3.0/24

14. Remove the **mgmt** subnet from **vnet2** and apply the changes.

Network Security Group (NSG) provisioning.

15. Comment lines **30** and **36** in **demo\main.tf** This enables a new module block that focusses of NSGs. Modules are not aware of each other or of the resources each creates. NSGs require Vnets and these are in a separate module. The root module must therefore act as an orchestrator, drawing information from the Vnet module and passing it to the NSG module. The VNet module must expose information by using outputs that the root module can then use when calling the NSG module.

Logic

modules\vnets\outputs.tf exposes the Vnet IDs it created

root\main.tf uses this information to populate its **nsgs** module call, looping for as many VNets that exist

modules\nsg\variables.tf is populated with this information

modules\nsg\main.tf deploys the NSGs based on these module variables

16. Run **terraform init** to initialize the new module

```
Initializing the backend...
Initializing modules...
- nsgs in modules\nsg
Initializing provider plugins...
- Reusing previous version of hashicorp/azurerm from the dependency lock file
- Using previously-installed hashicorp/azurerm v4.18.0
```

Note how the **nsg** module is now initialized

17. Run..

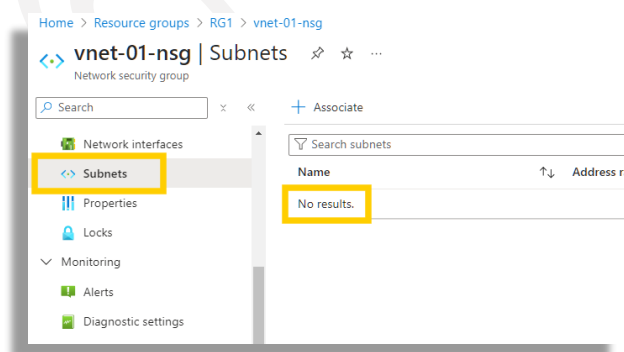
terraform plan

terraform apply --auto-approve

18. Run **terraform state list**

```
azurerm_resource_group.example
module.nsgs["vnet1"].azurerm_network_security_group.nsg
module.nsgs["vnet2"].azurerm_network_security_group.nsg
module.vnets["vnet1"].azurerm_subnet.subnet["subnet-01"]
module.vnets["vnet1"].azurerm_subnet.subnet["subnet-02"]
module.vnets["vnet1"].azurerm_virtual_network.vnet
module.vnets["vnet2"].azurerm_subnet.subnet["subnet-03"]
module.vnets["vnet2"].azurerm_subnet.subnet["subnet-04"]
module.vnets["vnet2"].azurerm_virtual_network.vnet
```

19. Use the Portal and verify that the NSGs exist but that our subnets are not associated with them...



NSG association provisioning

20. Comment lines **40** and **60** in **demo\main.tf**

```

resource "azurerm_subnet_network_security_group_association" "nsg_association" {
  for_each = merge([
    for vnet_key, vnet in module.vnets : {
      for subnet_name, subnet_id in vnet.subnets :
        "${vnet_key}-${subnet_name}" => {
          vnet_key = vnet_key
          subnet_id = subnet_id
        }
      }
    ]...)

  subnet_id              = each.value.subnet_id
  network_security_group_id = module.nsgs[each.value.vnet_key].nsg_id

  depends_on = [
    module.vnets,
    module.nsgs
  ]
}

```

Logic

From the output of the vnets module, loop over each VNet and extract its subnets.

Store the VNet key and Subnet ID in a structured map.

Build a unique for_each key using both values, for example vnet1-apps.

Flatten the generated maps using merge([...]).

Dynamically look up the correct NSG using each.value.vnet_key

21. Run..

terraform plan

terraform apply --auto-approve

22. Run **terraform state list**

```

PS C:\azure-tf-int\labs\04\demo> terraform state list
azurerm_resource_group.example
azurerm_subnet_network_security_group_association.nsg_association["vnet1-apps"]
azurerm_subnet_network_security_group_association.nsg_association["vnet1-data"]
azurerm_subnet_network_security_group_association.nsg_association["vnet2-apps"]
azurerm_subnet_network_security_group_association.nsg_association["vnet2-data"]
module.nsgs["vnet1"].azurerm_network_security_group.nsg
module.nsgs["vnet2"].azurerm_network_security_group.nsg
module.vnets["vnet1"].azurerm_subnet.subnet["apps"]
module.vnets["vnet1"].azurerm_subnet.subnet["data"]
module.vnets["vnet1"].azurerm_virtual_network.vnet
module.vnets["vnet2"].azurerm_subnet.subnet["apps"]
module.vnets["vnet2"].azurerm_subnet.subnet["data"]
module.vnets["vnet2"].azurerm_virtual_network.vnet
PS C:\azure-tf-int\labs\04\demo>

```

23. Switch to the Portal to verify the Subnet-NSG associations

Home > Resource Manager | Resource groups > RG1 > vnet-01-nsg

vnet-01-nsg | Subnets

Network security group

Search ◊ << + Associate

Overview
Activity log
Access control (IAM)
Tags
Diagnose and solve problems

Search subnets

Name	Address range	Virtual network
apps	10.0.1.0/24	vnet-01
data	10.0.2.0/24	vnet-01

Home > Resource Manager | Resource groups > RG1 > vnet-02-nsg

vnet-02-nsg | Subnets

Network security group

Search ◊ << + Associate

Overview
Activity log
Access control (IAM)
Tags
Diagnose and solve problems

Search subnets

Name	Address range	Virtual network
apps	10.1.1.0/24	vnet-02
data	10.1.2.0/24	vnet-02

Network Peering provisioning

24. Comment lines **62** and **74** in **demo\main.tf**

```
resource "azurerm_virtual_network_peering" "peerings" {
  for_each = {
    peer1to2 = { local = "vnet1", remote = "vnet2" }
    peer2to1 = { local = "vnet2", remote = "vnet1" }
  }

  name = each.key
  resource_group_name = azurerm_resource_group.example.name
  virtual_network_name = module.vnets[each.value.local].vnet_details.name
  remote_virtual_network_id = module.vnets[each.value.remote].vnet_details.id
}
```

resource "azurerm_virtual_network_peering" "peerings"

This defines a resource of type
azurerm_virtual_network_peering.

Instead of defining multiple individual resources, the **for_each** is used to dynamically create two peerings.

for_each = {...}

This is a **map** that contains two key-value pairs.

The **keys** (peer1to2, peer2to1) represent the names of the two peering relationships.

Each key maps to an **object** { local = "vnetX", remote = "vnetY" }, which holds the names of the local and remote virtual networks.

name = each.key

The each.key will be "peer1to2" for the first iteration and "peer2to1" for the second.

This dynamically sets the peering name based on the map keys.

resource_group_name

This stays the same for both peering resources since both virtual networks belong to the same resource group

module.vnets[each.value.local].vnet_details.name

each.value.local refers to the local virtual network for the current iteration.

On the first iteration (peer1to2), each.value.local = "vnet1", so it resolves to module.vnets["vnet1"].vnet_details.name.

On the second iteration (peer2to1), each.value.local = "vnet2", so it resolves to module.vnets["vnet2"].vnet_details.name.

module.vnets[each.value.remote].vnet_details.id

each.value.remote refers to the remote virtual network for the current iteration.

On the first iteration (peer1to2), each.value.remote = "vnet2", resolving to module.vnets["vnet2"].vnet_details.id.

On the second iteration (peer2to1), each.value.remote = "vnet1", resolving to module.vnets["vnet1"].vnet_details.id.

Logic

Terraform loops over the for_each map.

It creates two instances of azurerm_virtual_network_peering:

- One for peer1to2, linking vnet1 to vnet2.
- Another for peer2to1, linking vnet2 to vnet1.

Each instance gets dynamically populated with the correct name, virtual_network_name, and remote_virtual_network_id

25. Run..

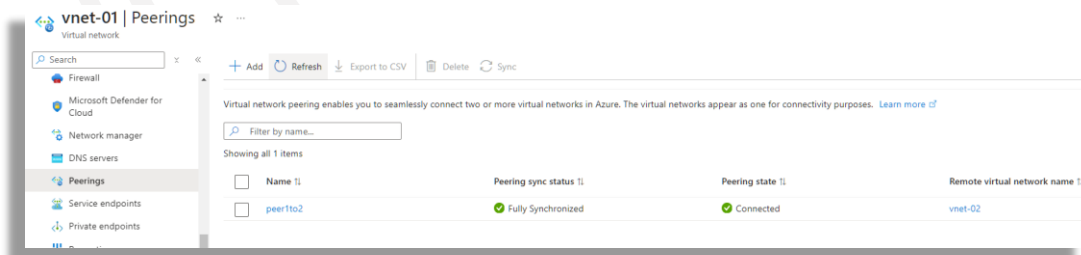
terraform plan

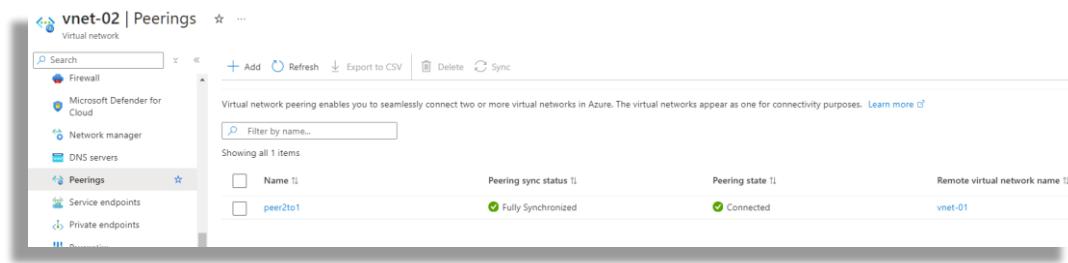
terraform apply --auto-approve

26. Run **terraform state list**

```
azurerm_resource_group.example
azurerm_subnet_network_security_group_association.nsg_association["subnet-01"]
azurerm_subnet_network_security_group_association.nsg_association["subnet-02"]
azurerm_subnet_network_security_group_association.nsg_association["subnet-03"]
azurerm_subnet_network_security_group_association.nsg_association["subnet-04"]
azurerm_virtual_network_peering.peer1to2
azurerm_virtual_network_peering.peer2to1
module.nsgs["vnet1"].azurerm_network_security_group.nsg
module.nsgs["vnet2"].azurerm_network_security_group.nsg
```

27. Confirm the peerings in the Portal





Lab Clean Up

1. Destroy all resources with **terraform destroy --auto-approve**

Congratulations, you have completed this lab

Copyright

© 2026 QA Michael Coulling-Green. All rights reserved. These lab materials are provided as part of an authorised training course. Course attendees may reuse these materials for their own personal learning and reference outside of the training session. Unauthorised copying, redistribution, publication, or use of these materials to deliver training is prohibited without prior written permission from the author.

© QA 2026